NomadReady PH — Directory Architecture and Claude Code Workflow Guide

1. Purpose of This Document

This document explains how the NomadReady PH hackathon team will structure the codebase, use Claude Code efficiently, and follow a spec-driven development workflow.

The goal is to avoid random AI-generated code and instead use Claude Code as a disciplined coding assistant that follows our approved specs, builds feature by feature, and avoids scope creep.

---

2. Development Mindset

We are not asking Claude Code to invent the product.

The product direction, scope, tech stack, scoring model, and MVP screens are already defined in the project specs.

Claude Code's job is to help execute the approved build plan.

The correct workflow is:

```
Spec → Task → Branch → Claude Implementation → Human Review → Test → Merge → Deploy
```

Not:

```
Prompt Claude → Build Randomly → Debug Chaos
```

---

3. Final Directory Architecture

We will use a monorepo structure.

This keeps backend, frontend, Docker, documentation, scripts, and deployment configuration in one place.

```
nomadready-ph/
  backend/
```

```
  manage.py
  requirements.txt
  Dockerfile
  config/
    settings.py
    urls.py
    asgi.py
    wsgi.py
  apps/
    destinations/
    listings/
    scoring/
    ai_advisor/
  tests/

frontend/
  package.json
  vite.config.js
  Dockerfile
  src/
    main.jsx
    App.jsx
    routes/
      Dashboard.jsx
      Assets.jsx
      MapView.jsx
      AIAdvisor.jsx
    components/
      layout/
      dashboard/
      assets/
      map/
      ai/
      ui/
    services/
      api.js
    lib/
      constants.js
      formatters.js

docs/
  00-product-context.md
  01-hackathon-scope.md
  02-build-spec.md
  03-directory-architecture-and-claude-workflow.md
  04-api-contract.md
  05-scoring-rules.md
  06-demo-script.md
```

```
    decisions/
      001-tech-stack.md
      002-scoring-model.md
      003-openai-advisor.md

  infra/
    deploy.md
    nginx.conf

  scripts/
    reset-dev.sh
    deploy.sh

  .github/
    workflows/
      ci.yml
      deploy.yml

  docker-compose.yml
  docker-compose.prod.yml
  .env.example
  README.md
  CLAUDE.md
```

---

4. Why This Architecture Works

`backend/`

Contains the Django backend.

Responsibilities:

- Store destination and local asset data
- Serve API endpoints
- Calculate readiness scores
- Integrate OpenAI API
- Store score snapshots and AI recommendations
- Seed Carles demo data

`frontend/`

Contains the React frontend.

Responsibilities:

- Display dashboard screens
- Show score cards and category breakdown
- Show local asset tables
- Show map pins
- Trigger AI Readiness Advisor
- Display AI-generated explanation and recommendations

`docs/`

Contains the project source of truth.

Claude Code must read the relevant docs before coding.

`infra/`

Contains deployment-related notes and server configuration.

`scripts/`

Contains helper scripts for development and deployment.

`.github/`

Contains CI/CD workflows.

`CLAUDE.md`

Defines how Claude Code should behave inside the repository.

This is one of the most important files in the project.

---

5. Backend Architecture

The backend uses:

- Django
- Django REST Framework
- PostgreSQL
- OpenAI API
- Docker

Backend apps:

```
backend/apps/
  destinations/
  listings/
  scoring/
  ai_advisor/
```

`destinations/`

Handles the Carles destination profile.

`listings/`

Handles local assets:

- Work spots
- Accommodations
- Services
- Transport
- Attractions

`scoring/`

Handles deterministic rule-based scoring.

This module calculates the numeric Digital Nomad Readiness Score.

OpenAI must not calculate the numeric score.

`ai_advisor/`

Handles OpenAI-powered explanation and recommendations.

This module uses the score and local data to generate an LGU-friendly action plan.

---

6. Frontend Architecture

The frontend uses:

- React
- Vite
- Tailwind CSS
- React Router
- Recharts
- Leaflet

- OpenStreetMap

Frontend routes:

```
/
/dashboard
/assets
/map
/ai-advisor
```

`/dashboard`

Shows:

- Overall readiness score
- Score label
- 5 category cards
- Top gaps
- Key metrics
- AI recommendations preview

`/assets`

Shows:

- Work spots
- Accommodations
- Services
- Transport
- Attractions
- Verification status

`/map`

Shows:

- Local asset pins
- Map legend
- Pin details popup

`/ai-advisor`

Shows:

- Score summary
- Generate AI Recommendations button
- AI readiness summary

- Strengths
- Weaknesses
- Recommended actions

---

7. Specs-Driven Development Rules

Before coding any feature, the developer and Claude Code must know:

1. What feature is being built
2. What spec defines it
3. What files are expected to change
4. What is out of scope
5. What acceptance criteria must pass

Claude Code should not begin coding immediately.

Claude Code should first read the relevant docs and explain its plan.

---

8. Core Docs Claude Must Read

Before implementing any feature, Claude Code should read:

```
CLAUDE.md
docs/02-build-spec.md
docs/04-api-contract.md
docs/05-scoring-rules.md
```

For demo polish, Claude should also read:

```
docs/06-demo-script.md
```

For directory or architectural decisions, Claude should read:

```
docs/03-directory-architecture-and-claude-workflow.md
docs/decisions/
```

---

9. Git Workflow

Use:

```
1 feature = 1 branch
```

Branch naming format:

```
feature/name-of-feature
```

Examples:

```
feature/project-setup
feature/backend-models-seed
feature/scoring-engine
feature/dashboard-overview
feature/assets-table
feature/map-view
feature/ai-readiness-advisor
feature/cicd-deployment
feature/demo-polish
```

Workflow:

1. Create feature branch from `main`
2. Ask Claude Code to read the specs and summarize the plan
3. Approve the plan
4. Let Claude Code implement only that feature
5. Run tests
6. Ask Claude Code to self-review against the spec
7. Human team lead reviews
8. Merge to `main` after approval
9. CI/CD deploys latest `main`

Rules:

- No direct push to `main`
- Main branch must always be deployable
- Unfinished work stays in feature branches
- One branch must solve one clear task
- No scope expansion without team lead approval

---

10. Claude Skills Needed for This Project

These are the Claude Code skills or working modes the team should use.

They may be implemented as formal Claude skills later, but the team can already follow them as prompting patterns.

10.1 Spec Reader Skill

Purpose:

Make Claude Code understand the task before coding.

Use this at the start of every feature branch.

Claude should:

- Read relevant docs
- Summarize the task
- List expected files to change
- List acceptance criteria
- Ask if anything is unclear
- Avoid coding until approved

10.2 Django Backend Builder Skill

Purpose:

Build Django backend features.

Use this for:

- Models
- Serializers
- Views
- URLs
- Services
- Management commands
- Backend tests

Rules:

- Keep views thin
- Keep scoring logic in `apps/scoring/services.py`
- Keep OpenAI logic in `apps/ai_advisor/services.py`
- Add tests for scoring
- Do not mix AI logic with numeric score calculation

10.3 React UI Builder Skill

Purpose:

Build frontend screens.

Use this for:

- Dashboard overview
- Asset table
- Map view
- AI Advisor page

Rules:

- Use Tailwind CSS
- Use reusable components
- Keep API calls in `frontend/src/services/api.js`
- Add loading states
- Add error states
- Do not hardcode data if API exists

10.4 Scoring Engine Implementer Skill

Purpose:

Implement the deterministic scoring system.

Rules:

- Only `lgu_verified` listings count toward scoring
- Draft listings do not increase scores
- Missing data lowers scores
- Category scores are capped at 100
- Overall score is rounded to the nearest whole number
- OpenAI must never calculate or modify the numeric score

10.5 OpenAI Advisor Implementer Skill

Purpose:

Build the AI Readiness Advisor.

Rules:

- Use OpenAI API only
- Give OpenAI structured score and listing data
- OpenAI explains the score
- OpenAI recommends LGU actions
- OpenAI must not invent places, services, or statistics
- OpenAI must not change the score
- Output should be structured JSON when possible

10.6 Test Writer Skill

Purpose:

Make sure the feature works.

Use this for:

- Backend scoring tests
- API response tests
- AI endpoint mock tests
- Manual frontend checklists

Rules:

- Backend logic needs tests
- Scoring rules must be tested
- AI endpoint should be testable with a mocked response
- Frontend must pass build

10.7 PR Reviewer Skill

Purpose:

Review code before merge.

Claude should check:

- Did this follow the spec?
- Did this modify unrelated files?
- Did this add unnecessary features?
- Do tests pass?
- Is the main branch still deployable?
- Are environment variables handled properly?

- Are secrets excluded from Git?

10.8 Demo Polish Skill

Purpose:

Improve the judge-facing experience without adding scope.

Use this near the end.

Rules:

- Improve labels
- Improve spacing
- Improve loading states
- Improve empty states
- Improve wording
- Improve demo clarity
- Do not add new modules
- Do not change architecture

---

11. Claude Plugins, Hooks, and Tools

For hackathon speed, keep tooling simple.

Use only what helps the team move faster without adding setup burden.

11.1 Useful Claude Code Tools

Use Claude Code for:

- Reading specs
- Editing files
- Running tests
- Reviewing branches
- Explaining errors
- Refactoring small sections
- Preparing demo polish

11.2 Hooks

Hooks can be used to run checks automatically after Claude edits files.

Suggested backend check:

```
docker compose exec backend python manage.py check
docker compose exec backend python manage.py test
```

Suggested frontend check:

```
docker compose exec frontend npm run build
```

Suggested final task report:

```
Files changed
What was implemented
Tests run
Acceptance criteria status
Known issues
```

11.3 Browser Testing

A browser or Playwright tool can be useful for UI testing.

Use it to check:

- Dashboard loads
- Asset filters work
- Map pins appear
- AI Advisor button works
- Layout looks good on a laptop screen

For the hackathon, manual browser testing is acceptable if setup time is limited.

11.4 GitHub or Bitbucket Integration

Use GitHub or Bitbucket integration if available.

Helpful for:

- Reading issues
- Reviewing pull requests
- Checking changed files
- Summarizing PRs

If integration setup takes too long, use normal Git commands and manual PR review.

---

12. How We Should Break Claude Tasks

This section is important. The quality of Claude Code output depends on the quality of the task prompt.

Do not give vague tasks.

Give Claude Code small, scoped, spec-based tasks.

Bad Task Example

```
Build the backend.
```

This is too broad. Claude may overbuild, create unnecessary files, or make architecture decisions without approval.

Better Task Example

```
On branch feature/backend-models-seed, create the Django models for Destination,
Listing, ScoreSnapshot, and AIRecommendation exactly as specified in docs/02-
build-spec.md.

Add migrations and a seed_demo_data management command for Carles demo data.

Do not build the scoring engine yet.
Do not build OpenAI integration yet.
Run Django checks after implementation.
```

Bad Task Example

```
Build the dashboard.
```

This is too broad.

Better Task Example

```
On branch feature/dashboard-overview, build the /dashboard React route using the
existing score API.

Show the overall score, score label, 5 category score cards, top gaps, and key
metrics.

Do not build the map page.
Do not build the AI Advisor page.
Use Tailwind CSS and existing API service patterns.
Run npm run build after implementation.
```

Good Claude Task Format

Use this structure:

```
You are working on branch: feature/[branch-name]

Read these files first:
- CLAUDE.md
- docs/02-build-spec.md
- docs/04-api-contract.md
- docs/05-scoring-rules.md

Task:
[Describe the exact task]

Scope:
[What is included]

Out of scope:
[What Claude must not build]

Expected files to modify:
[List files or folders]

Acceptance criteria:
[List pass/fail criteria]

Before coding:
Summarize your understanding and implementation plan.
Do not write code until the plan is approved.
```

13. Scaffolding Prompt for Claude Code

Use this prompt when starting the repository setup.

```
You are Claude Code working on the NomadReady PH hackathon MVP.

We are using a spec-driven methodology. Do not invent new architecture or
features.

First, read the following project rules carefully:

Project goal:
Build a hackathon MVP for NomadReady PH, an AI-powered LGU readiness dashboard
for digital nomad tourism.

Pilot destination:
Carles, Iloilo.

Final tech stack:
- Backend: Django, Django REST Framework, PostgreSQL
- Frontend: React, Vite, Tailwind CSS, React Router, Recharts, Leaflet,
OpenStreetMap
- AI: OpenAI API
- DevOps: Docker, Docker Compose, Git, CI/CD
- Deployment: server deployment from main branch

Important rule:
OpenAI must not calculate the numeric Digital Nomad Readiness Score.
The numeric score must be calculated by deterministic backend Python code.

Create the initial project scaffolding only.

Use this directory structure:

nomadready-ph/
  backend/
    manage.py
    requirements.txt
    Dockerfile
    config/
      settings.py
      urls.py
      asgi.py
      wsgi.py
    apps/
```

```
    destinations/
    listings/
    scoring/
    ai_advisor/
  tests/

frontend/
  package.json
  vite.config.js
  Dockerfile
  src/
    main.jsx
    App.jsx
    routes/
      Dashboard.jsx
      Assets.jsx
      MapView.jsx
      AIAdvisor.jsx
    components/
      layout/
      dashboard/
      assets/
      map/
      ai/
      ui/
    services/
      api.js
    lib/
      constants.js
      formatters.js

docs/
  00-product-context.md
  01-hackathon-scope.md
  02-build-spec.md
  03-directory-architecture-and-claude-workflow.md
  04-api-contract.md
  05-scoring-rules.md
  06-demo-script.md
  decisions/
    001-tech-stack.md
    002-scoring-model.md
    003-openai-advisor.md

infra/
  deploy.md
  nginx.conf
```

```
  scripts/
    reset-dev.sh
    deploy.sh

  .github/
    workflows/
      ci.yml
      deploy.yml

  docker-compose.yml
  docker-compose.prod.yml
  .env.example
  README.md
  CLAUDE.md

Scaffolding scope:
- Create the backend Django project structure.
- Create Django apps: destinations, listings, scoring, ai_advisor.
- Create the frontend React + Vite structure.
- Create placeholder route files for Dashboard, Assets, MapView, and AIAdvisor.
- Create Dockerfiles for backend and frontend.
- Create docker-compose.yml with backend, frontend, and db services.
- Create .env.example with required environment variables.
- Create placeholder docs files.
- Create placeholder scripts.
- Create basic README.md with local setup commands.
- Create initial CLAUDE.md placeholder.

Out of scope:
- Do not implement database models yet.
- Do not implement scoring yet.
- Do not implement OpenAI integration yet.
- Do not build UI details yet.
- Do not add login.
- Do not add the digital nomad public app.
- Do not add booking, reviews, payments, or multi-LGU features.

After scaffolding:
- Show the final directory tree.
- Explain what was created.
- List commands to run locally.
- List any setup issues or assumptions.
- Do not proceed to backend models until approved.
```

---

14. Feature Branch Build Order

Follow this order.

Branch 1: Project Setup

Branch:

```
feature/project-setup
```

Goal:

Create the base repo, Docker setup, Django project, React app, and placeholder docs.

Claude prompt:

Use the scaffolding prompt in Section 13.

Done when:

- Backend container runs
- Frontend container runs
- PostgreSQL container runs
- README has setup commands
- Directory structure matches this document

Branch 2: Backend Models and Seed Data

Branch:

```
feature/backend-models-seed
```

Goal:

Create backend models and Carles demo seed data.

Scope:

- Destination model
- Listing model
- ScoreSnapshot model

- AIRecommendation model
- Migrations
- `seed_demo_data` command

Out of scope:

- Scoring engine
- OpenAI integration
- Frontend UI

Branch 3: Scoring Engine

Branch:

```
feature/scoring-engine
```

Goal:

Implement deterministic scoring.

Scope:

- Scoring service
- Category score functions
- Overall score function
- Score label function
- Top gaps function
- Current score API
- Recalculate score API
- Backend tests

Out of scope:

- OpenAI recommendations
- Frontend dashboard polish

Branch 4: Dashboard Overview UI

Branch:

```
feature/dashboard-overview
```

Goal:

Build `/dashboard`.

Scope:

- Overall score card
- Score label
- 5 category score cards
- Top gaps
- Key metrics
- API integration

Out of scope:

- Map page
- AI Advisor page
- Assets table

Branch 5: Assets Table UI

Branch:

```
feature/assets-table
```

Goal:

Build `/assets`.

Scope:

- Listing table
- Category filters
- Verification status display
- API integration

Out of scope:

- Full CRUD
- Business portal
- Reviews

Branch 6: Map View

Branch:

```
feature/map-view
```

Goal:

Build `/map`.

Scope:

- Leaflet map
- OpenStreetMap
- Pins from listings
- Legend
- Popup details

Out of scope:

- Route planning
- Geocoding
- Google Maps

Branch 7: AI Readiness Advisor

Branch:

```
feature/ai-readiness-advisor
```

Goal:

Build OpenAI-powered readiness explanation and action plan.

Scope:

- OpenAI backend service
- AI advisor endpoint
- Frontend AI Advisor route
- Generate button
- AI output display

Out of scope:

- Letting OpenAI calculate the score
- Chatbot interface
- Public tourist-facing assistant

Branch 8: CI/CD and Deployment

Branch:

```
feature/cicd-deployment
```

Goal:

Deploy from main branch to server.

Scope:

- CI workflow
- Deploy workflow
- Docker production setup
- Server deployment commands
- Environment variable notes

Out of scope:

- Kubernetes
- Multi-server deployment
- Complex infrastructure

Branch 9: Demo Polish

Branch:

```
feature/demo-polish
```

Goal:

Make the hackathon demo smooth.

Scope:

- Better UI spacing
- Better labels
- Loading states
- Empty states
- Sample data labels
- Demo-friendly wording

Out of scope:

- New modules
- New screens
- New user types

---

15. Claude Code Setup Prompt for Every Feature

Use this before every feature implementation.

```
You are Claude Code working on NomadReady PH.

Current branch:
feature/[branch-name]

Read these first:
- CLAUDE.md
- docs/02-build-spec.md
- docs/03-directory-architecture-and-claude-workflow.md
- docs/04-api-contract.md
- docs/05-scoring-rules.md

Task:
[Insert exact feature task here]

Scope:
[Insert what is included]

Out of scope:
[Insert what must not be built]

Expected files or folders to modify:
[Insert expected files]

Acceptance criteria:
[Insert pass/fail criteria]

Before coding:
1. Summarize what you understand.
2. List the files you expect to modify.
3. List the implementation steps.
4. List any risks or unclear items.
5. Do not write code until I approve the plan.
```

---

16. Claude Code Completion Report Prompt

Use this after Claude finishes a feature.

```
Review your completed work against the approved spec.

Return a completion report with:

1. Feature branch name
2. Summary of what was implemented
3. Files changed
4. Tests or checks run
5. Acceptance criteria status
6. Anything not completed
7. Any risks or follow-up needed
8. Confirmation that no out-of-scope features were added
```

---

17. What Claude Must Not Build

Claude Code must not build these unless explicitly approved:

- Full digital nomad public app
- Login system unless required for deployment
- Business owner portal
- Booking system
- Payment system
- Review system
- Real-time Wi-Fi testing
- Public destination pages
- Multi-LGU comparison
- Advanced admin panel
- Mobile app
- Chatbot interface
- Complex role-based permissions
- Kubernetes or microservices

The hackathon MVP must stay focused.

---

18. Team Usage Guidance

Use Claude Code this way:

Team Lead uses Claude for planning, reviewing, and demo polish.

Backend dev uses Claude for Django, scoring, APIs, tests.

Frontend dev uses Claude for React screens, Tailwind, map, and API integration.

Claude should follow the docs.

Claude should not decide scope.

Claude should not change architecture without approval.

Claude should not add features because they are “nice to have.”

Claude should implement the approved specs, one feature branch at a time.

---

19. Final Project Principle

The winning MVP is simple:

```
Local tourism data → Readiness score → Gaps → Map → AI action plan
```

The team must protect this simplicity.

The final demo message is:

NomadReady PH helps LGUs prepare for the future of tourism by showing whether a destination is ready for digital nomads, what gaps need to be fixed, and what actions should be taken first.

Pilot destination:

Carles, Iloilo — from island tourism destination to digital nomad-ready destination.